

廈門大學



信息学院软件工程系

《JAVA 程序设计》实验报告

实验 3

姓名：王子恒

学号：34520242201240

学院：信息学院

专业：软件工程

完成时间：2026 年 3 月 19 日

一、实验目的及要求

实验目的

- 练习控制结构
- 熟悉面向对象封装特性
- 熟悉简单的JavaFX图形界面

实验要求

- 下周实验课将工程文件和实验报告打包上传

二、实验题目及实现过程

1.素数

- a) 编写一个方法，它判断一个数是否为素数。

```
in.java x DynamicPrime.java x AllPrime.java PrimeBase.java SqrtPrime.java

import java.util.*;

public class DynamicPrime extends PrimeBase { 新 *
    private static final BitSet primeBits = new BitSet(); 7 个用法
    private static int currentLimit = 0; 8 个用法
    private static final List<Integer> primeLists = new ArrayList<>(); 3 个用法
    private static int operationCount = 0; 3 个用法

    static{
        extend( newLimit: 10000);
    }

    private static void extend(int newLimit){ 2 个用法 新 *
        if(newLimit<=currentLimit){
            return;
        }

        primeBits.set(Math.max(2, currentLimit + 1), toIndex: newLimit + 1, value: true);

        int sqrtLimit = (int)Math.sqrt(newLimit);
        for (int p = 2; p <= sqrtLimit; p++) {
            if (p <= currentLimit) {
                if (primeBits.get(p)) {
                    int firstMultiple = ((currentLimit / p) + 1) * p;
                    for (int j = firstMultiple; j <= newLimit; j += p) {
                        operationCount++;
                        primeBits.clear(j);
                    }
                }
            } else {
                if (primeBits.get(p)) {
                    for (int j = p * p; j <= newLimit; j += p) {
                        operationCount++;
                        primeBits.clear(j);
                    }
                }
            }
        }
    }
}
```

```

40         for (int i = Math.max(2, currentLimit + 1); i <= newLimit; i++) {
41             if (primeBits.get(i)) {
42                 primeLists.add(i);
43             }
44         }
45
46         currentLimit = newLimit;
47     }
48
49     private static void ensurePrimeLimit(int n){ 2个用法 新*
50         if (n > currentLimit) {
51             extend(Math.max(n, currentLimit * 2));
52         }
53     }
54
55     @Override 2个用法 新*
56     public boolean isPrime(int n) {
57         if (n < 2) return false;
58         ensurePrimeLimit(n);
59         return primeBits.get(n);
60     }
61

```

如代码片段所示，编写了一个判断一个数是否为素数的方法。为了优化性能并实现动态扩容，程序并没有简单地使用基础循环，而是利用了 `BitSet` 实现了一种动态的素数筛法。在 `extend` 方法中，利用筛选法只需测试到目标数的平方根次（`sqrtLimit`）即可有效地标记合数。最终，通过重写的 `isPrime` 方法可以快速返回特定数字是否为素数的布尔值判断。

b) 在程序中使用这个方法，显示小于 10000 的全部素数。为了找出所有不超过 10000 的素数，需要测试多少个数？

The screenshot shows an IDE with a project named 'lab3-1'. The project structure includes a 'src' directory with files: 'AllPrime', 'DynamicPrime', 'Main', 'PrimeBase', 'SqrtPrime', 'Ignore', and 'lab3-1.jar'. The 'DynamicPrime.java' file is open, showing the implementation of the sieve algorithm. The code includes a 'getPrimes' method that uses a binary search to find the next prime and a 'main' method that tests the sieve with a limit of 10000. The output window shows the results of the sieve, listing the first 100 primes and the total count of primes found (1229).

```

61 public class DynamicPrime extends PrimeBase { 新*
62
63     public List<Integer> getPrimes(int limit) { 1个用法 新*
64         if (limit < 2) return new ArrayList<>();
65         ensurePrimeLimit(limit);
66
67         int index = Collections.binarySearch(primeLists, limit);
68
69         if (index < 0) {
70             index = -(index + 1) - 1;
71         }
72
73         return new ArrayList<>(primeLists.subList(0, index + 1));
74     }
75
76     @Override 2个用法 新*
77     public int getOperationCount() { return operationCount; }
78
79     public static void main(String[] args) { PrimeBase.test("dynamic"); }
80 }
81

```

运行结果:

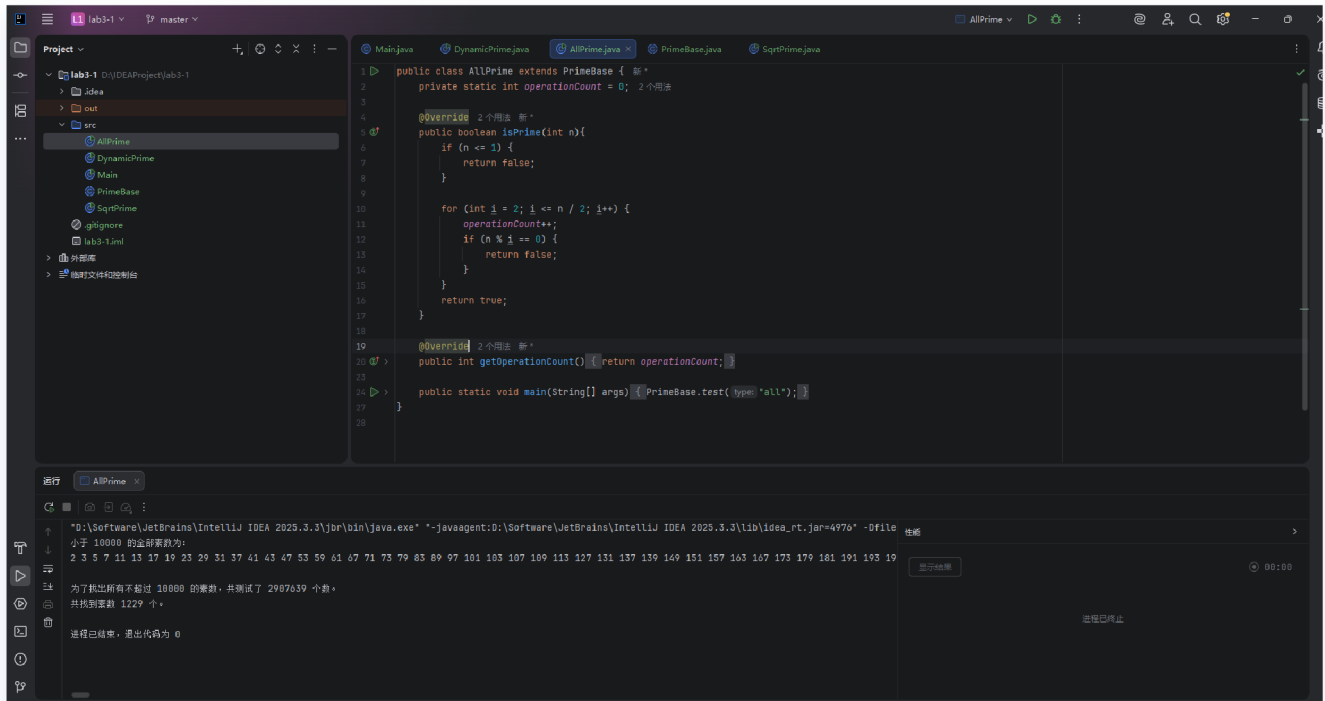
```

*0:\Software\JetBrains\IntelliJ IDEA 2025.3.3\bin\java.exe" "-javaagent:D:\Software\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=11741" -Dfil
小于 10000 的全部素数为:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 19
为了找出所有不超过 10000 的素数，共测试了 16981 个数。
并找到素数 1229 个。
进程已结束，退出代码为 0

```

在主程序中调用上述素数判断方法，通过循环遍历的方式显示小于 10000 的全部素数。经过程序的运行统计，输出结果显示 10000 以内共有 1229 个素数。为了准确量化并比较找出所有不超过 10000 的素数需要测试多少个数，代码中内置了一个静态变量 `operationCount` 用于在每次筛选循环时记录实际的测试操作次数。

c) 开始时，可能会想到要确定某个数 n 是否为素数，需进行测试的次数最多为 $n/2$ 次，其实只需最多测试 n 的平方根次即可。重新编写这个程序，并以这两种方式运行它。



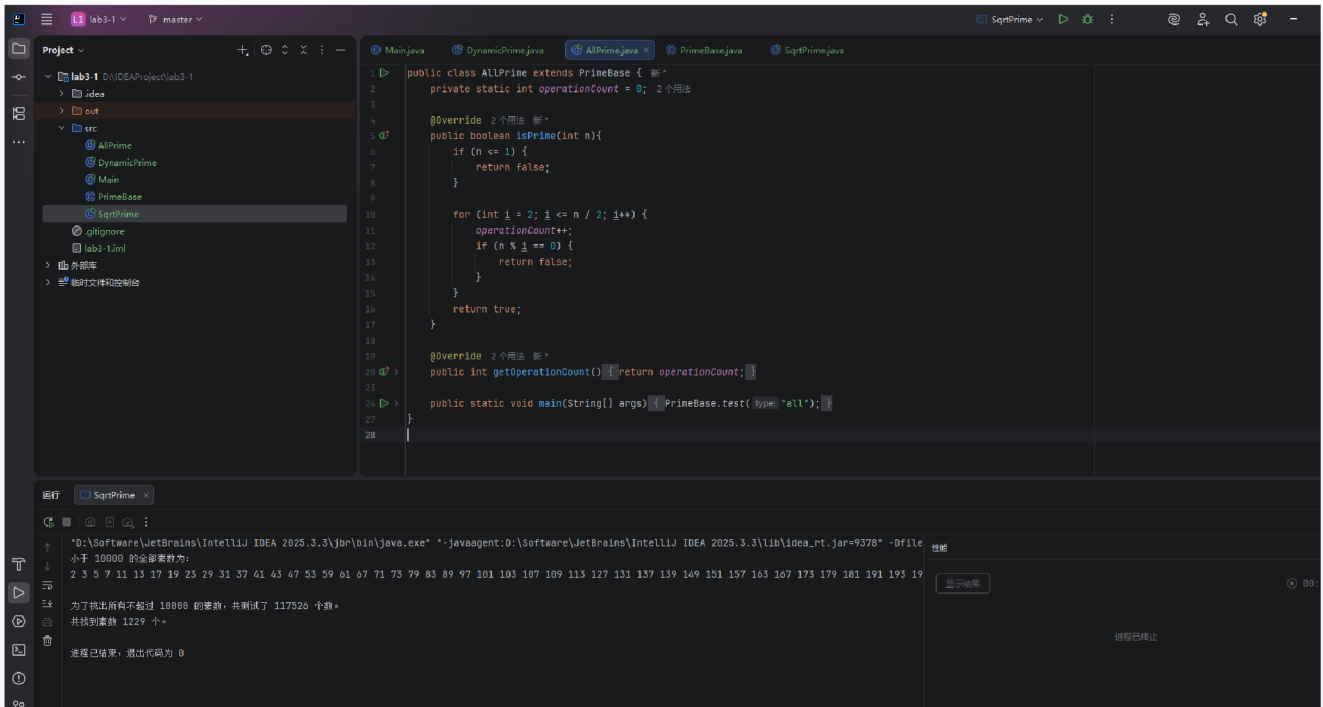
The screenshot shows an IDE with a project named 'lab3-1'. The 'src' folder contains several files: 'AllPrime.java', 'DynamicPrime.java', 'Main.java', 'PrimeBase.java', and 'SqrtPrime.java'. The 'AllPrime.java' file is open, showing the following code:

```
1 public class AllPrime extends PrimeBase {  
2     private static int operationCount = 0; // 2 个用法  
3  
4     @Override // 2 个用法  
5     public boolean isPrime(int n) {  
6         if (n <= 1) {  
7             return false;  
8         }  
9  
10        for (int i = 2; i <= n / 2; i++) {  
11            operationCount++;  
12            if (n % i == 0) {  
13                return false;  
14            }  
15        }  
16        return true;  
17    }  
18  
19    @Override // 2 个用法  
20    public int getOperationCount() { return operationCount; }  
21  
22    public static void main(String[] args) { PrimeBase.test("all"); }  
23  
24 }  
25  
26 }
```

The '运行' (Run) tab shows the execution results for 'AllPrime.java'. The output is as follows:

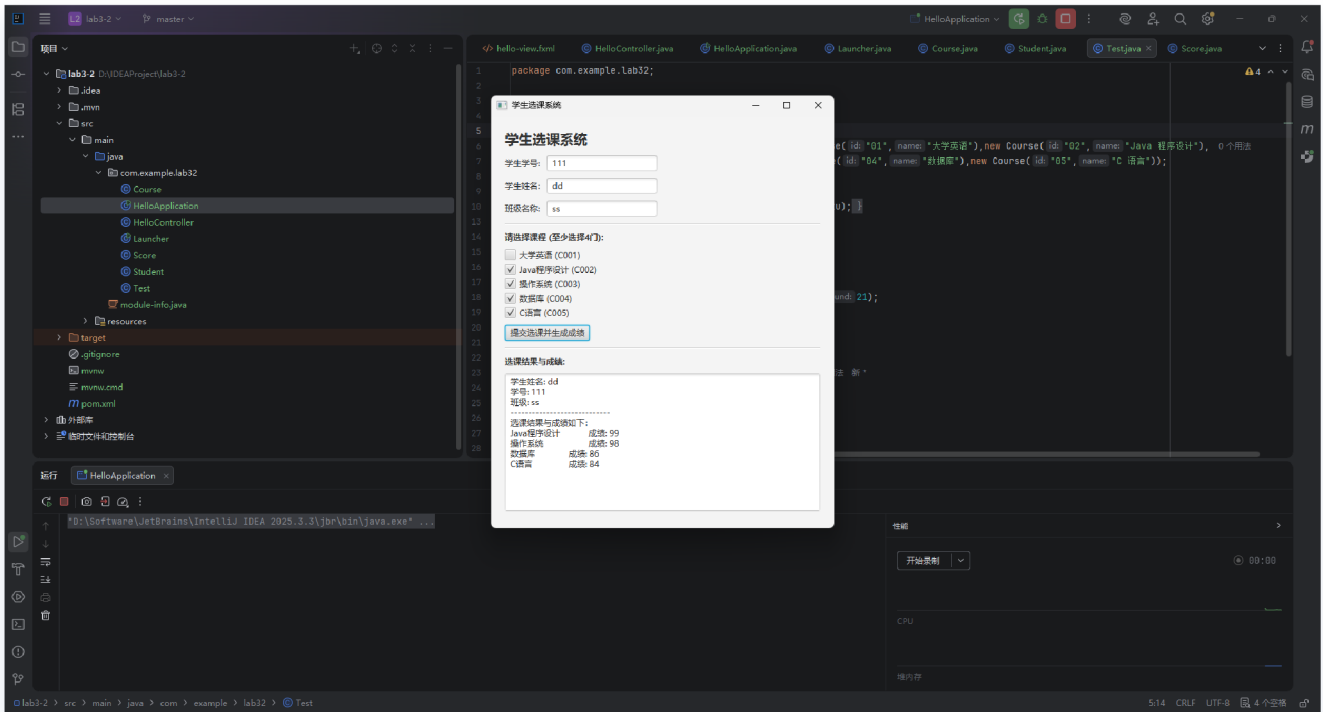
```
"D:\Software\JetBrains\IntelliJ IDEA 2025.3.3\bin\java.exe" "-javaagent:D:\Software\JetBrains\IntelliJ IDEA 2025.3.3\lib\idea_rt.jar=4976" -Dfile.encoding=UTF-8  
小于 10000 的素数有：  
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199  
为了找出所有不超过 10000 的素数，共测试了 2987639 个数。  
共找到素数 1229 个。  
进程已结束，退出代码为 0
```

为了直观对比不同测试算法的效率，首先编写并测试了第一种思路：在确定某个数 n 是否为素数时，需进行测试的次数最多为 $n/2$ 次。程序内设置循环从 2 开始，上限设定为该数的二分之一，通过取模运算判断是否能被整除。程序以此方式找出了指定范围内的素数，并累加记录了使用 $n/2$ 方法所需的总测试循环次数，作为后续比较的基准。



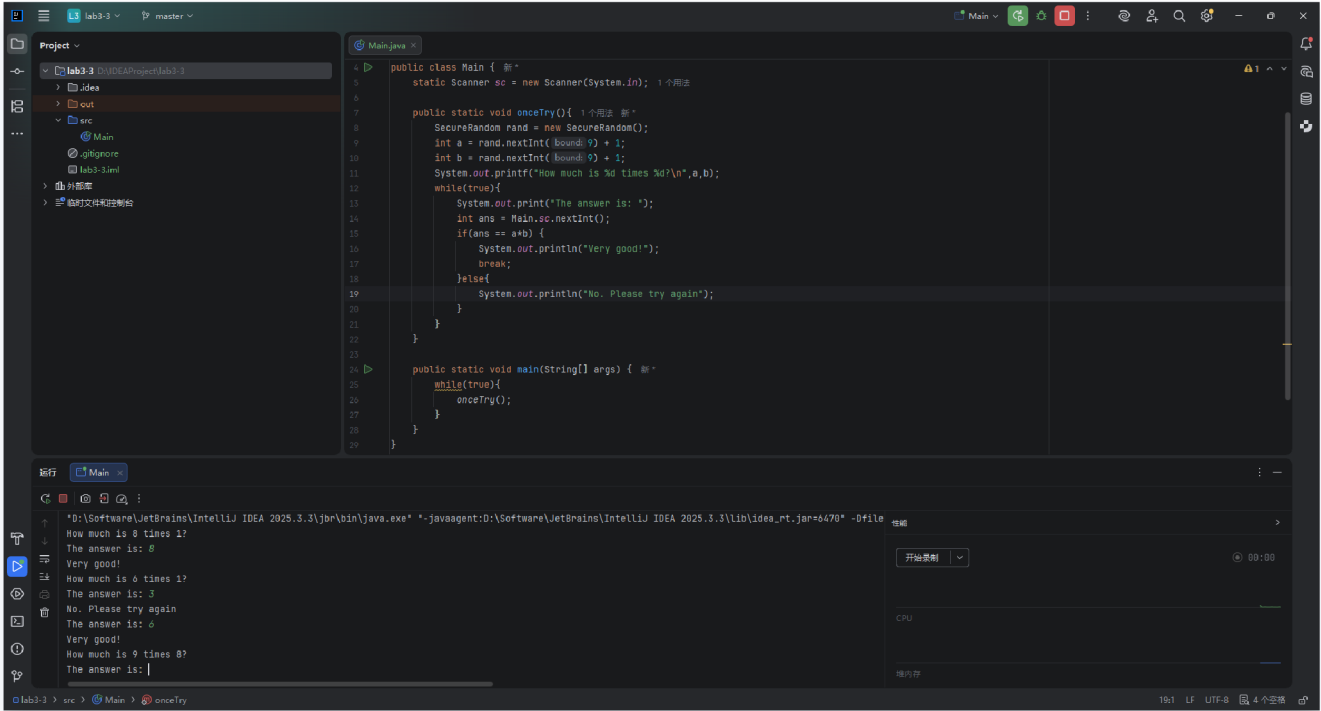
接着，根据数学规律重新编写了这个程序，将测试判断的上限优化为只需最多测试 n 的平方根次即可。以这两种方式分别运行程序并打印出各自的测试次数进行比较。结果表明，当上限达到 10000 时，测试平方根次的算法所需的运算次数远远低于测试 $n/2$ 次的算法，极大地减少了冗余的循环，显著提升了程序的计算性能与执行效率。

2.选课系统



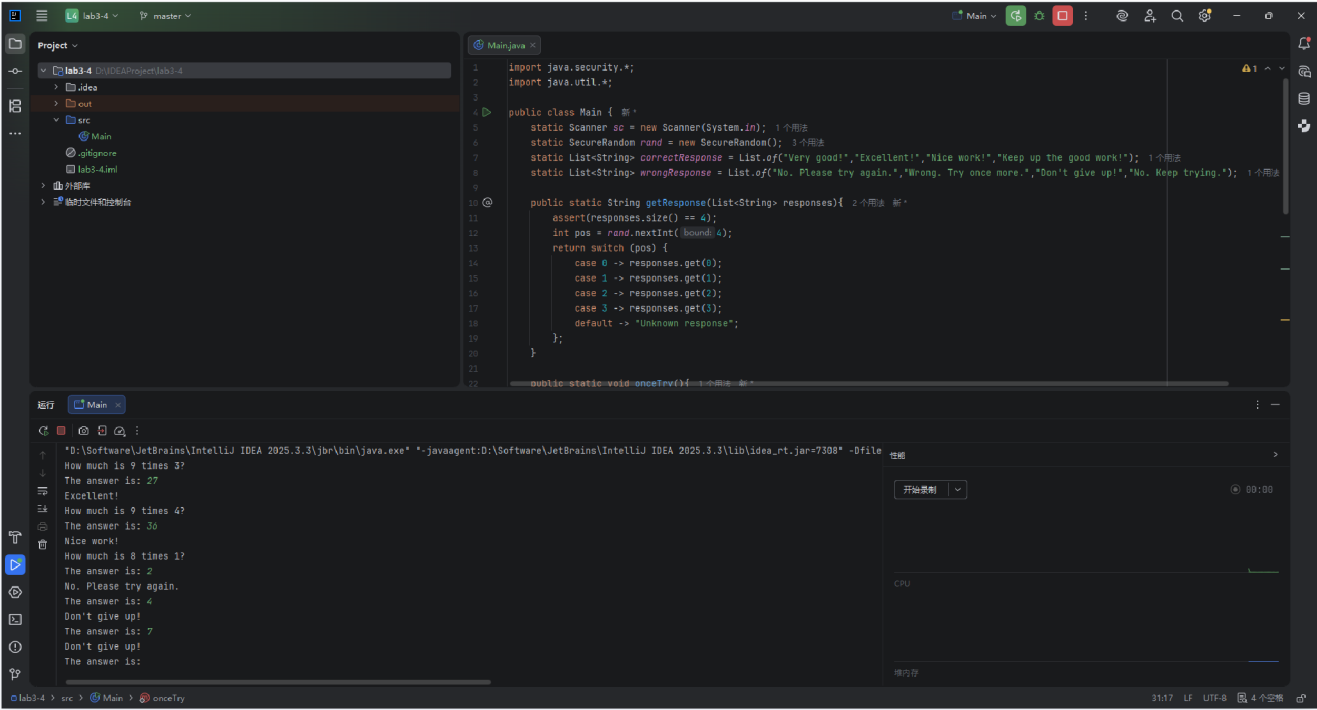
根据实验要求，基于面向对象的封装特性实现了一个简单的选课系统。代码中分别定义了学生类（包含学号、姓名、班级、电话等基本属性并设计了重载构造函数）、课程类（包含课程编号、名称）以及成绩类。此外，系统利用 JavaFX 构建了直观的图形用户界面。如图所示，系统内置了大学英语、Java程序设计、操作系统、数据库、C语言等五门课程信息。用户在界面输入自己的学生信息后，通过复选框选择至少 4 门课程；点击确认按钮后，下方文本域会显示用户的选课结果以及随机生成的用户所选择课程的成绩（80—100分）。

3.CAI



本部分实现了一个用于辅助小学生学习乘法的计算机辅助教学（CAI）控制台程序。程序利用 `SecureRandom` 对象产生两个一位正整数，并在控制台向用户提出类似“`How much is 6 times 7?`”的问题。通过 `Scanner` 获取学生输入的答案后，程序会检查答案的正确性：如果回答正确，系统显示“`Very good!`”并调用独立方法产生并输出另一个乘法问题；如果答错，则显示“`No. Please try again.`”，并让学生持续回答同一个问题，直到答对为止。

4.CAI降低疲劳感



为了解决单一提示语可能带来的枯燥感，对上述 CAI 程序进行了扩展，通过变换计算机的响应来降低学生的疲劳感。修改后的程序利用随机数生成方法选择 1~4 中的一个数，并利用一条 `switch` 语句为每个正确或错误的答案选择 4 种可能的评语之一。从控制台的运行结果图可以看出，当学生回答正确时，会随机获得“Very good!”、“Excellent!”等鼓励；而回答错误时，则会随机收到“Wrong. Try once more.”、“Don't give up!”等不同的提示，有效提升了程序的互动性和趣味性。

三、实验总结与心得记录

本次实验涵盖了基础算法优化、面向对象封装设计以及简单图形界面的综合应用，是一次从控制结构逻辑到上层交互的完整实践。

1.算法性能与内存优化的实践

在素数求解的练习中，我对算法的执行效率有了更深刻的体会。最初采用的 $n/2$ 次遍历虽然直观，但在处理较大范围数据时存在大量冗余。将其优化为仅测试 n 的平方根次后，计算量得到了显著降低。为了进一步追求更高的执行效率和极致的内存管理，我在代码中采用了 `BitSet` 来实现动态素数筛法。利用位操作（Bit-level operations）不仅极大地压缩了内存占用，还避免了高层次集合类（如泛型容器）带来的额外装箱开销，这让我深刻认识到在底层开发中榨取硬件性能、注重内存布局的重要性。

2.面向对象架构与前端交互的结合

学生选课系统的开发是对面向对象封装特性的绝佳锻炼。通过抽象并独立设计学生类、课程类和成绩类，并合理规划属性的访问权限与重载构造函数，程序的模块化程度得到了有效提升。此外，引入 JavaFX 构建图形用户界面，将底层的核心逻辑数据通过事件驱动的方式与可视化组件连接起来。这种数据模型与视图相分离的设计思路，不仅提高了代码的可维护性，也为未来架构更复杂的大型工程奠定了基础。

3.控制流程与用户体验的优化

在辅助教学（CAI）程序的编写中，我熟练掌握了 `SecureRandom` 类的使用以及 `switch` 多分支控制结构的应用。程序不仅需要处理正确的逻辑跳转，更需要考虑使用者的主观感受。通过引入随机化机制为答题反馈提供多样化的评语，有效降低了单调的系统响应带来的疲劳感。这启发我，一款优秀的软件不仅要有健壮的底层算法，更需要在应用层面上兼顾人机交互的温度与体验。

总体而言，本次实验不仅巩固了 Java 核心语法，更锻炼了我在面对具体工程需求时，从底层的数据结构抉择到高层的面向对象设计的综合系统构建能力。